Further benchmarking for railway routing and scheduling in Flaspland

Ryan Murphy

University of Potsdam, Germany

Abstract. The benchmarking experiments from our original paper left us with a clear set of tasks. Among those were two experiments regarding environment representation and configuration settings. In these new experiments, we have found that the SubNodes environment representation performs better than the others regardless of the encoding. We have also found no significant improvement by selecting a new configuration setting.

1 Introduction

Our original paper introduced the Flaspland toolkit [3], which handles railway routing and scheduling problems in answer set programming (ASP). In addition to presenting a formalization for Flatland [2], we defined three graphs (SubNodes, EdgFuncs, HypEdges) that serve as alternative representations to the given environment, and we provided encodings (DriveMap, DifCons) that solve routing and scheduling problems using these graph structures. Initial benchmarking experiments on these encodings left us with a number of key takeaways: (1) graph representations offered various advantages; (2) the SubNodes graph yielded the best overall performance of all representations for DriveMap; (3) running the DifCons encoding led to significant improvements to computation time.

2 Experiments

The benchmarking process from the original paper left us with a clear path forward. This paper addresses two new benchmarking tasks:

1. Running the DifCons encoding with other graph representations
2. Employing alternative clingo configuration settings

2.1 Running DifCons with other graph representations

Originally, the DifCons encoding was tested using only the SubNodes graph representation. This experiment tests the DifCons encoding on the EdgFuncs and HypEdges graph representations. **Results.** SubNodes outperformed the other two. In our largest environments, SubNodes dedicated 50 seconds on average to grounding, whereas HypEdges and EdgFuncs needed 131 and 152, respectively. The discrepancy is more pronounced when reviewing overall times, which include solving. In

our largest environments, SubNodes needed 52 seconds on average to complete, whereas HypEdges and EdgFuncs needed 570 and 997, respectively. A similar trend is observed in the smaller environments.

2.2 Employing alternative clingo configuration settings

There are several configuration settings that were designed to suit clingo for different types of problems. The configuration settings in `version 5.8.0` are `frumpy`, `jumpy`, `tweety`, `handy`, `crafty`, and `trendy`. The original benchmarking experiment using the DifCons and SubNodes graph was run once per configuration.
Results. Selecting an alternative configuration setting had no measurable benefit on the performance of the encoding under these circumstances. Only one configuration setting, `frumpy`, performed better than the default setting. However, the change is not significant enough to justify a future deviation from the default setting.

3 Discussion

The results of the first experiment leave us with a few points worth discussing. First, why does SubNodes perform so much better in general than the other two? The data demonstrate that the biggest influence on performance came from the selection of the graph representation. SubNodes outperformed all other representations regardless of encoding; HypEdges likewise outperformed EdgFuncs regardless of encoding. Respectively within each graph selection, DifCons performed better than DriveMap.

Second, the data show that the grounding sizes, in terms of rules, variables, and constraints, are smaller when using SubNodes. However, this fails to explain how, in the original experiment, DifCons with SubNodes had relatively small solving times, with respect to all other results. This possibly is due to the grounding required in the formation of the graphs themselves, but might also have something to do with how the validity of paths is checked for each representation.

Going forward, future benchmarking steps include increasing the number of trains in environments and increasing the size of the environments themselves, to test how well our encodings scale in larger settings. Additionally, we see potential in implementing an approach seen in [1], wherein time steps are removed from the path finding process. The SubNodes representation tolerates repeated visits to a cell without violating acyclicity conditions.

References

[1] J. Behrens et al. "Routing and Scheduling in Answer Set Programming applied to Multi-Agent Path Finding: Preliminary Report". In: *CoRR* abs/2403.12153 (2024). URL: `https://arxiv.org/abs/2403.12153`.

[2] S. Mohanty et al. "Flatland-RL : Multi-Agent Reinforcement Learning on Trains". In: *CoRR* (2020). URL: `https://arxiv.org/abs/2012.05893`.

[3] R. Murphy et al. “Flaspland: A Testbed for Routing and Scheduling in ASP”. In: 2026.